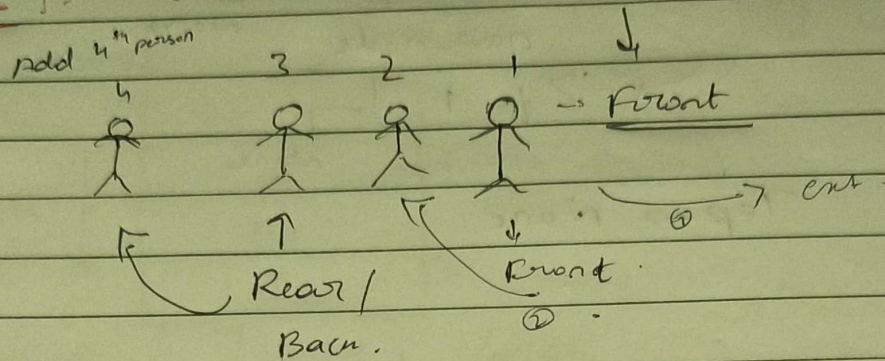
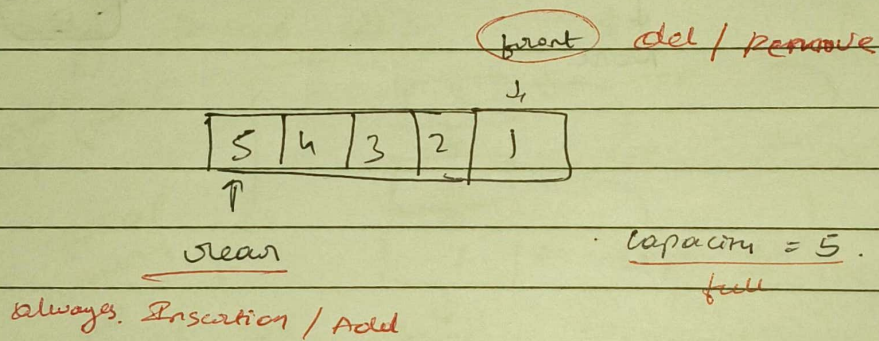


13 Introduction to Queue

Queue : (FIFO)



When Front
↑
Rear } comes on same index
called
Empty Queue.



→ Rear :- Adding / Insertion (Enqueue)

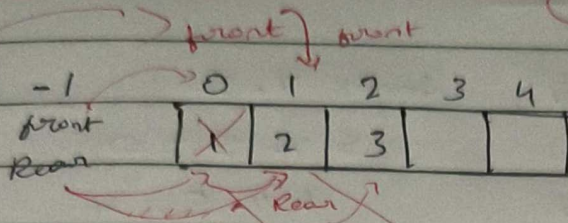
→ Front :- Removing / Del (Dequeue)

→ capacity = max.

→ Front == Rear } Empty.

Peek → Value.

Queue Implementation using Array:

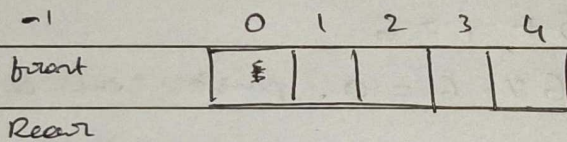
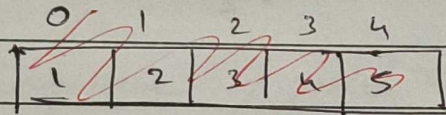


enqueue (1)

enqueue (2)

enqueue (3)

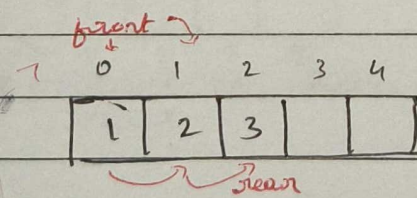
dequeue ()



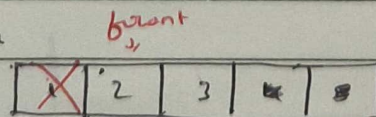
enqueue (1)

enqueue (2)

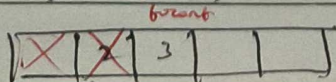
enqueue (3)



dequeue ()

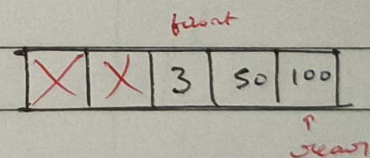


dequeue ()



enqueue (50)

enqueue (100)



Capacity = 5, max - 1 = 4

enqueue (200) → X fail

When all delete & front & rear come to same index then it will insert front but this has the drawback

Modulus % Operator :

modules of ~~6~~ 5. Index % 5.

remainder

$$5 \% 5 = 0 \rightarrow$$

0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5

$$4 \% 5 = 4$$

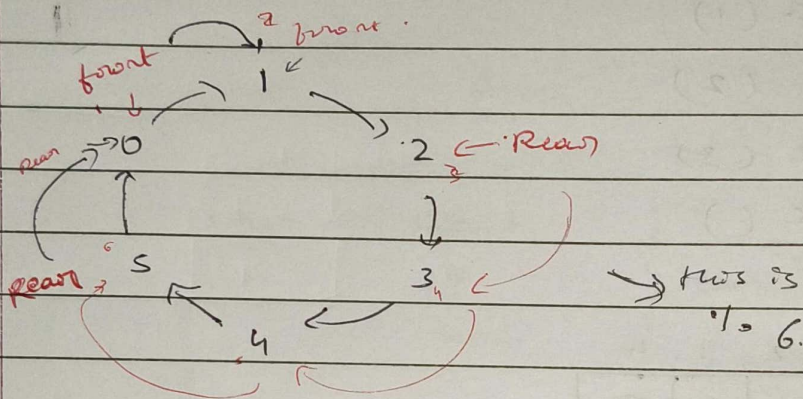
$$3 \% 5 = 3$$

$$1 \% 5 = 1$$

$$0 \% 5 = 0$$

$$10 \% 5 = 0$$

$$11 \% 5 = 1$$



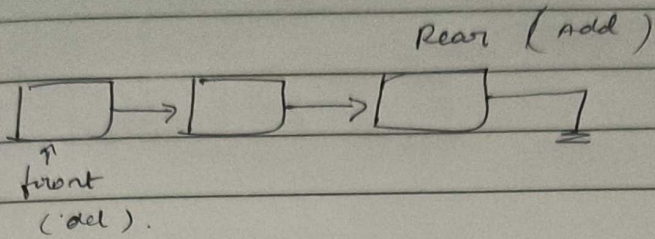
$\rightarrow$ when del one

$\rightarrow$ add one.

$$5 + 1 = 6$$

$6 \% 6 = 0$, push the 0 after 5.

Singly linked list in Queue :-



DMA $\rightarrow$ heap memory

Queue in Python :- (FIFO)

$\rightarrow$ Enqueue / Add

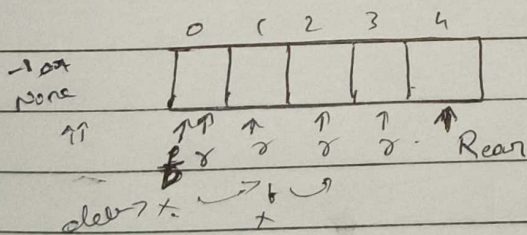
$\rightarrow$ Dequeue / Remove

$\rightarrow$ Peek ()

$\rightarrow$ Count ()

$\rightarrow$ Print ()

1 $\rightarrow$ Array / List

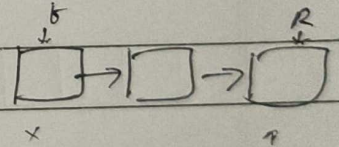


enqueue $O(1)$

dequeue $O(n)$

#2 Collection
deque

#3 Custom Implⁿ
using Singly linked list



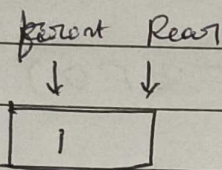
$O(1)$

$O(1)$

Using SLL :-

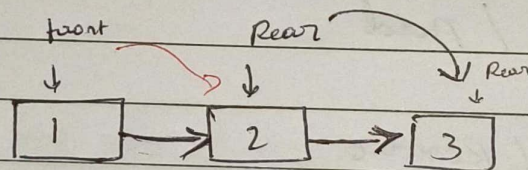
front Rear
↓ ↓
None.

Adding 1

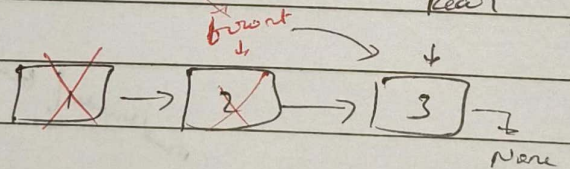


Enqueue / Insert:

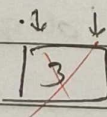
Adding 2/3



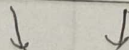
for dequeue



front Rear



front Rear



None.


```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Queue:
7
8     def __init__(self):
9
10        self.rear = None
11        self.front = None
12        self.count = 0
13
14    def enqueue(self, data):
15        new_node = Node(data)
16
17        # queue is empty
18        # front → [1] → [2] → [3] → None : front is stay 1st only
19        # rear -----^                  only rear is trverses
20
21        if self.rear is None:
22            self.front = new_node
23            # self.rear = new_node
24
25        else:
26            self.rear.next = new_node
27            # self.rear = new_node
28
29        self.rear = new_node
30        self.count += 1
31        print(f"element {data} is inserted in queue")
32
33    def dequeue(self):
34        # queue is empty
35        if self.front is None:
36            print("queue is empty")
37            return -100

```

```
6 class Queue:
32     def dequeue(self):
37
38         return_data = self.front.data
39         self.front = self.front.next
40
41         if self.front is None:
42             self.rear = None
43
44         self.count -= 1
45         print(f"Removing element {return_data} from the queue")
46         return return_data
47
48     def peek(self) -> int:
49
50         if self.front is None:
51             print("queue is empty")
52             return -100
53
54         print("peek is", self.front.data)
55         return self.front.data
56
57     def get_count(self) -> int:
58         print("count is ", self.count)
59         return self.count
60
61     def print_all_nodes(self):
62
63         if self.front is None:
64             print("queue is empty")
65             return -100
66
67         print("the queue elements are ")
68         current_node = self.front
69
70         while current_node is not None:
71             print(current_node.data, end = " -> ")
```



```
69
70     while current_node is not None:
71         print(current_node.data, end = " -> ")
72         current_node = current_node.next
```

```
73
74     print("None")
75
```

```
76
77 que = Queue()
78 que.dequeue()
79 que.print_all_nodes()
```

```
80
81 que.enqueue(1)
82 que.print_all_nodes()
83 que.dequeue()
84 que.print_all_nodes()
```

```
85
86 que.enqueue(1)
87 que.enqueue(2)
88 que.enqueue(3)
```

```
89
90 que.print_all_nodes()
91 que.get_count()
92 que.peek()
93 que.dequeue()
94 que.print_all_nodes()
```

```
95
96
97
98
```